

EMT Disconnected One-Point Building Blocks

Quark Loops, Gluon Loops, Stochastic Sources, and HDF5 Outputs

PyQUDA Measurement Notes

July 15, 2026

Contents

1	Purpose and File Map	2
2	Conventions	2
3	Quark One-Point Workflow	2
3.1	Target Trace	2
3.2	Stochastic Trace Estimator	3
3.3	Z_n Noise	3
3.4	Quark EMT Operator Implemented in Code	4
3.5	Symmetrization	5
3.6	Noise Average and Volume Normalization	5
3.7	Identity Bilinear and Flowed-Noise Norm	5
3.8	Ringed-Fermion Kinetic Normalization	6
3.9	Gradient-Flow Schedule for Quarks	6
3.10	Four-Dimensional Fermion Flow and the Time-Slice Projector	7
3.11	Why a Single Initial-Time Projector Is Not the Current Estimator	8
4	Hierarchical Probing and Source Bookkeeping	8
4.1	Ordering Choices	8
4.2	Bookkeeping Datasets	9
5	Quark HDF5 Layout	9
5.1	Raw-Bilinear Analysis Example	10
5.2	Storage Weight of One Canonical EMTc	10
5.3	Base-Oriented Production, Checkpoints, and Finalization	11
6	Gluon One-Point Workflow	12
6.1	Clover Field Strength	12
6.2	Implemented Gluon EMT Loop	13
6.3	Gradient-Flow Schedule for Gluons	13
7	Gluon HDF5 Layout	13
8	Disconnected Combination with Proton Two-Point Functions	14
9	End-to-End Proton Diagnostic Workflow	14
10	How to Read Smoke Tests	15
11	Recommended Next Algorithmic Upgrade	16

1 Purpose and File Map

This note documents the hadron-independent one-point functions used to build disconnected contributions to proton and pion energy-momentum tensor (EMT) matrix elements. The main implementation is

`pyquda_measurement_utils/Disconnected_1pt_EMT_vibe_develop.py`.

The Perlmutter application entry points are in

`application/EMT_disconnected_1pt/perlmutter`.

The shorter command-oriented workflow guide is

`docs/EMT_disconnected_1pt/EMT_proton_disconnected_guide.md`.

The one-point workflows do not form hadron three-point functions by themselves. They measure vacuum loop building blocks. A disconnected hadron matrix element is formed later by correlating these one-point loops with hadron two-point functions on the same gauge ensemble.

2 Conventions

We work in Euclidean spacetime with four directions. Mathematical formulas use $\mu, \nu, \rho = 1, 2, 3, 4$, while Python loops use zero-based indices $\mu, \nu, \rho = 0, 1, 2, 3$. The temporal direction is the fourth Euclidean direction. PyQUDA stores local lattice fields in even-odd layout; the displayed formulas suppress this storage detail.

For a spatial momentum transfer q , the code constructs spatial phases with `MomentumPhase.getPhases`. In this note the phase is written as

$$\Phi_q(x) = e^{i\mathbf{q}\cdot\mathbf{x}}, \quad (1)$$

where only the spatial coordinates are Fourier transformed and the time coordinate t is kept explicit.

The spatial volume normalization used in the averaged HDF5 datasets is

$$V_3 = L_x L_y L_z. \quad (2)$$

The raw per-source quark datasets are stored before this V_3 normalization. The averaged quark and gluon datasets are volume-normalized.

3 Quark One-Point Workflow

The quark workflow is usually the first part to study. It contains the stochastic trace estimator, the hierarchical-probing source bookkeeping, the χ -type diagnostics, and the flowed quark EMT loop used in disconnected analysis.

3.1 Target Trace

A disconnected quark loop has the schematic form

$$L_\Gamma^q(q, t; t_f) = \sum_{\mathbf{x}} \Phi_q(x) \text{Tr}_{sc}[\Gamma[U(t_f)](x) D^{-1}[U](x, x)], \quad (3)$$

where D^{-1} is the quark propagator, Γ is the local operator kernel, and Tr_{sc} denotes spin-color trace. Directly computing all diagonal entries of $D^{-1}(x, x)$ is too expensive, so the trace is estimated with stochastic sources.

3.2 Stochastic Trace Estimator

Let ξ_r be a stochastic source vector whose spin-color-site components satisfy

$$\langle \xi_r \rangle_{\text{noise}} = 0, \quad \langle \xi_r(i) \xi_r^\dagger(j) \rangle_{\text{noise}} = \delta_{ij}. \quad (4)$$

Here i, j are combined site, spin, and color indices. The code obtains

$$\eta_r = D^{-1} \xi_r \quad (5)$$

from the Dirac inverter. For any operator matrix A ,

$$\langle \xi_r^\dagger A \eta_r \rangle_{\text{noise}} = \langle \xi_r^\dagger A D^{-1} \xi_r \rangle_{\text{noise}} \quad (6)$$

$$= \sum_{i,j,k} A_{ij} (D^{-1})_{jk} \langle \xi_{r,i}^\dagger \xi_{r,k} \rangle_{\text{noise}} \quad (7)$$

$$= \sum_{i,j} A_{ij} (D^{-1})_{ji} = \text{Tr}(A D^{-1}). \quad (8)$$

This is the Hutchinson trace-estimator logic used by the quark one-point workflow.

3.3 Z_n Noise

The ordinary stochastic scheme is selected with

$$\text{noise_scheme} = \text{zn}. \quad (9)$$

Each component of ξ is drawn independently from

$$\xi(i) \in \left\{ \exp\left(\frac{2\pi i r}{n}\right) \mid r = 0, 1, \dots, n-1 \right\}, \quad (10)$$

where $n = \text{n_zn}$. The EMT quark one-point workflow now defaults to full-volume $\mathbb{Z}_4$ noise. Its counter-based generator hashes the global four-dimensional coordinate, spin, color, configuration number, base-noise index, and optional stream salt. Consequently the global source is reproducible and independent of the chosen MPI decomposition.

This counter construction fixes a serious failure mode of rank-local backend random-number generators. If every MPI rank resets an ordinary generator to the same state s , ranks with the same local lattice shape draw the same array. For a rank-local index ℓ ,

$$\xi_r(\ell) = F(s, \ell) = \xi_{r'}(\ell), \quad r \neq r'. \quad (11)$$

The corresponding distinct global sites $x_r + \ell$ and $x_{r'} + \ell$ then have unit rather than vanishing noise covariance,

$$\mathbb{E}_\xi[\xi^*(x_r + \ell) \xi(x_{r'} + \ell)] = 1, \quad r \neq r', \quad (12)$$

so the required identity in Eq. (4) is not sampled. This creates rank-periodic off-diagonal terms, can bias the stochastic trace, and changes the global source when the MPI decomposition changes. Mixing the rank into the seed removes exact repetition only for one fixed process layout; it still makes the source depend on the rank assignment. The production counter instead uses physical global coordinates and semantic keys, so the same configuration, base index, and stream reconstruct exactly the same global field for every MPI geometry. Legacy backend-RNG loop data are therefore not combined with counter-noise production data.

3.4 Quark EMT Operator Implemented in Code

The quark one-point kernel now measures the complete sixteen-element PyQUDA Dirac basis Γ_A . For each source it saves the local and one-derivative primitive bilinears

$$L_A^{(r)}(q, t; t_f) = \sum_{\mathbf{x}} \Phi_q(x) \xi_r^\dagger(x; t_f) \Gamma_A \eta_r(x; t_f), \quad (13)$$

$$L_{A\mu}^{D,(r)}(q, t; t_f) = -\frac{1}{2} \sum_{\mathbf{x}} \Phi_q(x) \xi_r^\dagger(x; t_f) \Gamma_A [D_{+\mu} - D_{-\mu}] \eta_r(x; t_f). \quad (14)$$

The gamma axis is ordered as

5, T, T5, X, X5, Y, Y5, Z, Z5, I, SXT, SXY, SXZ, SYT, SYZ, SZT

and the derivative axis is X, Y, Z, T. The sixteen gamma contractions are batched after each derivative is constructed, so this extension changes contraction and I/O cost but does not add inversions, fermion-flow updates, or covariant-derivative applications.

All EMT programs import this basis from `pyquda_measurement_utils/fermion_bilinear_basis.py`. In the DeGrand–Rossi representation returned by the current PyQUDA implementation, the four Euclidean generators are

$$\gamma_1 = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}, \quad \gamma_2 = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}, \quad (15)$$

$$\gamma_3 = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix}, \quad \gamma_4 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad (16)$$

and

$$\gamma_5 = \gamma_1 \gamma_2 \gamma_3 \gamma_4 = \text{diag}(1, 1, -1, -1). \quad (17)$$

The resulting HDF5 Gamma axis is not numerical-ID order:

Index	Label	Raw matrix	Index	Label	Raw matrix
0	5	γ_5	8	Z5	$\gamma_3 \gamma_5$
1	T	γ_4	9	I	1
2	T5	$-\gamma_4 \gamma_5$	10	SXT	$\gamma_1 \gamma_4$
3	X	γ_1	11	SXY	$\gamma_1 \gamma_2$
4	X5	$\gamma_1 \gamma_5$	12	SXZ	$\gamma_1 \gamma_3$
5	Y	γ_2	13	SYT	$\gamma_2 \gamma_4$
6	Y5	$-\gamma_2 \gamma_5$	14	SYZ	$\gamma_2 \gamma_3$
7	Z	γ_3	15	SZT	$\gamma_3 \gamma_4$

The corresponding PyQUDA IDs are

$$[15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12]. \quad (18)$$

The numerical matrices themselves are stored in `gamma_matrices`; an analysis should read the file metadata rather than reproduce the table by hand.

The EMT is a derived vector-channel observable. Defining $B_{\nu\mu} = L_{\gamma_{\nu,\mu}}^D$, the previous per-source, pre-volume-normalized object remains

$$L_{\nu\mu}^{q,(r)}(q, t; t_f) = -\frac{1}{2} \sum_{\mathbf{x}} \Phi_q(x) \xi_r^\dagger(x; t_f) \gamma_\nu [D_{+\mu} - D_{-\mu}] \eta_r(x; t_f). \quad (19)$$

It is reconstructed directly from the primitive array. In code the underlying batch contraction is equivalent channel by channel to

```

tmp = U_f.pure_gauge.covDev(eta, mu) - U_f.pure_gauge.covDev(eta, mu + 4)
field_A = contract("...ia,Aij,...ja->A...", xi.data.conj(), gamma_A, tmp.data)
L_D[:, mu] = -0.5 * project_all_gamma_channels(field_A)

```

The first tensor index is ν , because the gamma matrix γ_ν is applied after choosing the derivative direction μ . This is why the code fills `Tmunu[nu, mu]` before symmetrization.

The raw matrices deliberately retain the historical PyQUDA bit-mask basis used by the qTMD and two-point workflows. A stored matrix `physical_from_pyquda` defines

$$\Gamma_A^{\text{phys}} = \sum_B C_{AB} \Gamma_B^{\text{raw}}. \quad (20)$$

With $\gamma_5 = \gamma_1 \gamma_2 \gamma_3 \gamma_4$, the raw and physical axial labels agree for `X5` and `Z5`, while

$$\Gamma_{Y5}^{\text{phys}} = -\Gamma_{Y5}^{\text{raw}}, \quad \Gamma_{T5}^{\text{phys}} = -\Gamma_{T5}^{\text{raw}}. \quad (21)$$

The six raw tensor matrices satisfy

$$\Gamma_{S\mu\nu}^{\text{raw}} = \gamma_\mu \gamma_\nu = \frac{1}{2} [\gamma_\mu, \gamma_\nu] \quad (\mu < \nu). \quad (22)$$

If the Hermitian Euclidean convention $\sigma_{\mu\nu} = \frac{i}{2} [\gamma_\mu, \gamma_\nu]$ is desired, the saved tensor channel must be multiplied by i . Recording this distinction avoids silently assigning inconsistent axial signs or tensor phases in later form factor analyses.

3.5 Symmetrization

The EMT tensor is symmetrized after all unsymmetrized components have been accumulated:

$$L_{\mu\nu}^q \leftarrow \frac{1}{2} (L_{\mu\nu}^q + L_{\nu\mu}^q). \quad (23)$$

The code then sets both entries equal. The HDF5 writer stores only the upper triangle $\mu \leq \nu$, with dataset names `T11`, `T12`, $\dots$, `T44`. The lower triangle should be reconstructed by symmetry if needed.

3.6 Noise Average and Volume Normalization

For N_{eff} effective sources, the averaged quark one-point loop is

$$L_{\mu\nu, \text{avg}}^q(q, t; t_f) = \frac{1}{V_3} \frac{1}{N_{\text{eff}}} \sum_{r=1}^{N_{\text{eff}}} L_{\mu\nu}^{q, (r)}(q, t; t_f). \quad (24)$$

The code stores the raw primitive data and their volume-normalized averages; the symmetric EMT is saved only as a small averaged derived view:

$$\text{raw/derivative_bilinear_pervec} \leftrightarrow L_{A\mu}^{D, (r)}(q, t; t_f), \quad (25)$$

$$\text{avg/Tmunu/Tij} \leftrightarrow L_{ij, \text{avg}}^q(q, t; t_f). \quad (26)$$

3.7 Identity Bilinear and Flowed-Noise Norm

The identity scalar bilinear and the flowed-source norm are

$$\chi_0^{(r)}(q, t; t_f) = \sum_{\mathbf{x}} \Phi_q(x) \xi_r^\dagger(x; t_f) \eta_r(x; t_f), \quad (27)$$

$$\chi_1^{(r)}(q, t; t_f) = \sum_{\mathbf{x}} \Phi_q(x) \xi_r^\dagger(x; t_f) \xi_r(x; t_f). \quad (28)$$

χ_0 is already the identity channel of the complete local basis and is not duplicated. χ_1 checks the noise normalization. In code:

```
scalar_trace = local_bilinear[gamma_list.index("I")]
```

```
dot_xi_xi = contract("etzyxbc,etzyxbc->etzyx", xi.data.conj(), xi.data)
flowed_noise_norm = impose_P_Breit_slice(dot_xi_xi, phases_3pt)
```

Schema version 3 stores only the latter separately as `raw/flowed_noise_norm_pervec` and `avg/flowed_noise_norm`; both averages use the same V_3 normalization as the primitive bilinears.

3.8 Ringed-Fermion Kinetic Normalization

The flowed-noise norm is not the ringed-fermion normalization. The kinetic observable is already contained in the zero-momentum diagonal quark EMT components. For each effective stochastic source r , the implementation forms

$$K_r(t_f, \tau) = -\frac{2}{V_3} \sum_{\mu=1}^4 L_{\mu\mu,r}^q(q=0, \tau; t_f) = \frac{1}{V_3} \sum_{\mu, \mathbf{x}} \xi^\dagger(x; t_f) \gamma_\mu [D_{+\mu} - D_{-\mu}] \eta(x; t_f). \quad (29)$$

No new inversion, fermion-flow update, covariant derivative, or MPI gather is needed: Eq. (29) is evaluated directly from the four diagonal vector channels of `raw/derivative_bilinear_pervec`. The momentum list must contain exactly one $q=0$ entry for every EMT quark measurement.

Every EMTc stores this data in `derived/ringed/kinetic_pervec` and `derived/ringed/kinetic_spacetime_avg`. Publishing one HDF5 path with one atomic rename removes the former two-file failure window. No ringed factor is stored: a factor computed from one configuration is not the final ensemble normalization.

For configuration c , the stored spacetime average is

$$K_c(t_f) = \frac{1}{N_{\text{eff}} N_t} \sum_{r, \tau} K_{r,c}(t_f, \tau). \quad (30)$$

The analysis must first form the gauge-ensemble mean

$$K_{\text{code}}(t_f) = \langle K_c(t_f) \rangle_{\text{cfg}}, \quad (31)$$

and only then, for a single-flavor trace in fundamental $SU(3)$, evaluate

$$Z_{\text{ring}}^{(f)}(t_f) = \frac{-2N_c}{(4\pi)^2 t_f^2 K_{\text{code}}^{(f)}(t_f)}. \quad (32)$$

In particular, $\langle 1/K_c \rangle_{\text{cfg}} \neq 1/\langle K_c \rangle_{\text{cfg}}$ in general, so configuration-local inverse factors must not be averaged. If instead the kinetic expectation value has already been summed or averaged over N_f degenerate flavors, the numerator and denominator must use the same flavor convention. A quark EMT bilinear made from ringed fields receives one factor Z_{ring} , because the square roots from χ and $\bar{\chi}$ multiply to this single bilinear factor. The $t_f = 0$ output step is kept as an unflowed diagnostic and must be excluded from Eq. (32). The standalone `application/flowed_quark_ringed_norm` workflow remains available for dedicated high-statistics and spin-color-diluted calculations. It uses the same base/HP-part files and fingerprinted base-level text sample log as EMT. Production resume trusts this log without probing HDF5, so a logged base may be transferred immediately. Finalization is performed where all parts are collected and checks each part once while streaming the canonical file.

3.9 Gradient-Flow Schedule for Quarks

The quark workflow measures before flowing. Therefore step 0 is the unflowed observable. If $\epsilon = \text{flow_epsilon}$, then

$$t_f(n) = n\epsilon, \quad n = 0, \dots, \text{flow_steps}. \quad (33)$$

The first flow interval is internally split into ten smaller updates:

```

for step in range(Nsteps + 1):
    measure primitive bilinears and flowed_noise_norm
    if step == 0:
        flow [xi, eta] for 10 substeps of stepsize/10
    elif step < Nsteps:
        flow [xi, eta] for 1 step of stepsize

```

The stochastic source ξ and solution η are flowed together as one `MultiField`. This keeps both fields on the same flowed gauge background. After all flow steps are measured, the code extracts `derived/ringed/kinetic.pervect` algebraically from the stored $q = 0$ vector derivative diagonals in the same `EMTc`.

3.10 Four-Dimensional Fermion Flow and the Time-Slice Projector

Let $K(t_f)$ denote the gauge-covariant fermion-flow kernel from flow time zero to t_f . Suppressing source labels, the fields measured by the code are

$$\xi_f = K(t_f)\xi, \quad \eta_f = K(t_f)D^{-1}\xi. \quad (34)$$

The fermion flow is four dimensional: its covariant Laplacian couples all four Euclidean directions, including time. Its characteristic smoothing radius is approximately $\sqrt{8t_f}$. This radius is a diffusion scale, not a strict compact-support boundary.

Let P_τ project the measured output field onto the absolute Euclidean time slice τ , while leaving spatial, spin, and color indices unchanged. For an EMT kernel Γ on the flowed gauge background, one stochastic measurement can be written as

$$\begin{aligned} \widehat{L}(\tau, t_f) &= \xi_f^\dagger P_\tau \Gamma \eta_f \\ &= \xi^\dagger K^\dagger(t_f) P_\tau \Gamma K(t_f) D^{-1} \xi. \end{aligned} \quad (35)$$

Using the full-volume noise identity in Eq. (4),

$$\begin{aligned} \mathbb{E}_\xi[\widehat{L}(\tau, t_f)] &= \text{Tr} \left[K^\dagger(t_f) P_\tau \Gamma K(t_f) D^{-1} \right] \\ &= \text{Tr} \left[P_\tau \Gamma K(t_f) D^{-1} K^\dagger(t_f) \right]. \end{aligned} \quad (36)$$

The projector P_τ means that the physical observable remains a spatial sum at a resolved insertion time; Eq. (36) is not a sum over insertion time. Nevertheless, the initial source must cover the full four-dimensional vector space because $K^\dagger P_\tau \Gamma K D^{-1}$ connects the measured time slice to source components at other times.

Equivalently, define

$$A_\tau = K^\dagger(t_f) P_\tau \Gamma K(t_f) D^{-1}. \quad (37)$$

For finite noise count, the off-diagonal stochastic contribution is

$$\widehat{L}(\tau, t_f) - L(\tau, t_f) = \sum_{i \neq j} \xi_i^* (A_\tau)_{ij} \xi_j. \quad (38)$$

The left index is constrained by the flowed time-slice operator, but the right index can originate from any Euclidean time. These terms vanish in the noise expectation; their finite-sample size is a variance question.

3.11 Why a Single Initial-Time Projector Is Not the Current Estimator

Suppose one replaces the full-volume source by only one time-diluted source $\xi_d = P_d \xi$. At nonzero flow time its expectation value is

$$\mathbb{E}_\xi \left[\xi^\dagger P_d K^\dagger P_\tau \Gamma K D^{-1} P_d \xi \right] = \text{Tr} \left[P_d K^\dagger P_\tau \Gamma K D^{-1} P_d \right], \quad (39)$$

which is generally not Eq. (36). Four-dimensional fermion flow spreads the endpoint time slice over several initial-time components, so choosing only $d = \tau$ omits contributions.

Time dilution is not invalid in principle. For a complete orthogonal family

$$\sum_d P_d = \mathbf{1}, \quad P_d P_{d'} = \delta_{dd'} P_d, \quad (40)$$

summing all diluted estimators reconstructs the full trace:

$$\sum_d \text{Tr} [P_d A_\tau P_d] = \text{Tr} A_\tau. \quad (41)$$

It requires all projectors and additional inversions. The present EMT workflow therefore uses one full-volume base source per stochastic solve and retains all absolute insertion times. In the unflowed limit $t_f = 0$, where $K(0) = \mathbf{1}$, a local loop at τ can be obtained from the matching exact-time projector; this special case does not justify dropping the other projectors for the finite-flow measurements stored in the same file.

4 Hierarchical Probing and Source Bookkeeping

The quark one-point workflow supports

$$\text{noise_scheme} \in \{\text{zn}, \text{hierarchical_probing}\}. \quad (42)$$

Hierarchical probing starts from a full-volume base noise vector ξ_b and multiplies it by deterministic site-only Rademacher vectors $h_k(x) = \pm 1$:

$$\xi_{b,k}(x) = h_k(x) \xi_b(x). \quad (43)$$

The probing vectors are designed to reduce near-diagonal contributions to the stochastic trace variance. In the current implementation the number of HP vectors must be a positive power of two:

$$\text{hp_num_vectors} = 1, 2, 4, 8, \dots \quad (44)$$

The effective number of inversions is

$$N_{\text{eff}} = \begin{cases} \text{n_vec}, & \text{noise_scheme}=\text{zn}, \\ \text{n_vec} \times \text{hp_num_vectors}, & \text{noise_scheme}=\text{hierarchical_probing}. \end{cases} \quad (45)$$

This value is stored as `effective_n_inversions`.

4.1 Ordering Choices

Four HP orderings are currently implemented:

Ordering	Meaning
<code>global_xyz_t_gray_projected_to_evenodd</code>	Gray-code the full x, y, z, t site id.
<code>interleaved_xyz_t_binary_projected_to_evenodd</code>	Interleave binary bits from x, y, z, t .
<code>interleaved_xyz_binary_projected_to_evenodd</code>	Interleave only spatial bits; the HP sign is time independent.
<code>spatial_xyz_then_t_gray_projected_to_evenodd</code>	Gray-code spatial sites first, then append time bits.

For the spatial ordering,

$$h_k(x, y, z, t) = h_k(x, y, z), \quad (46)$$

whereas a four-dimensional ordering has signs that also vary with t . Both cases multiply the same full-volume base noise and remain nonzero on every time slice; neither is time dilution. Spatial HP devotes all available probing bits to spatial locality, while isotropic 4D HP can suppress temporal off-diagonal noise at the cost of allocating fewer early bits to space. Their efficiency is observable dependent and should be compared at fixed inversion cost.

4.2 Bookkeeping Datasets

For each effective source, the output records:

Dataset	Meaning
<code>raw/source_index</code>	Effective source index after HP expansion.
<code>raw/base_noise_index</code>	Original base noise index.
<code>raw/hp_index</code>	HP vector index used for that base noise.

For example, with `n_vec = 1` and `hp_num_vectors = 2`, the expected bookkeeping is

$$\text{source_index} = [0, 1], \quad \text{base_noise_index} = [0, 0], \quad \text{hp_index} = [0, 1]. \quad (47)$$

5 Quark HDF5 Layout

The production measurement writes validated base/HP shards and the explicit finalizer streams them into the canonical file. The main raw datasets are

$$\text{raw/local.bilinear_pervec} : [N_{\text{eff}}, 16, N_q, N_{\text{steps}} + 1, N_t], \quad (48)$$

$$\text{raw/derivative.bilinear_pervec} : [N_{\text{eff}}, 16, 4, N_q, N_{\text{steps}} + 1, N_t], \quad (49)$$

$$\text{raw/flowed_noise_norm_pervec} : [N_{\text{eff}}, N_q, N_{\text{steps}} + 1, N_t]. \quad (50)$$

The averaged datasets are

$$\text{avg/local.bilinear} : [16, N_q, N_{\text{steps}} + 1, N_t], \quad (51)$$

$$\text{avg/derivative.bilinear} : [16, 4, N_q, N_{\text{steps}} + 1, N_t], \quad (52)$$

$$\text{avg/Tmunu/T11, T12, \dots, T44} : [N_q, N_{\text{steps}} + 1, N_t], \quad (53)$$

$$\text{avg/flowed_noise_norm} : [N_q, N_{\text{steps}} + 1, N_t]. \quad (54)$$

Only $\mu \leq \nu$ tensor components are written in the averaged `avg/Tmunu` group.

The canonical loop file is source independent:

$$\text{EMTc}/\langle \text{lat} \rangle . \text{EMTc} . \langle \text{cfg} \rangle . \langle \text{ama} \rangle . \langle \text{sm} \rangle . \text{h5}.$$

It stores every absolute insertion time and is reused for all hadron two-point source times on the same configuration. Before forming a disconnected three-point product, analysis applies `roll(time_axis, -source.t)` to both quark and gluon loops. Thus output index τ_{rel} represents $\tau_{\text{abs}} = (t_0 + \tau_{\text{rel}}) \bmod N_t$.

The embedded kinetic-only schema is

```
derived/ringed/kinetic_pervec
derived/ringed/kinetic_spacetime
```

EMT uses full spin-color noise rather than point spin-color dilution. The group attributes record the unique zero-momentum index, the $-2/V_3$ relation, and that ringed factors are not stored. `ringed_factors_stored=False` distinguishes this subset from a full standalone output. The counter-noise configuration, stream, algorithm, HP ordering, Dirac parameters, gauge pre-processing, boundary condition, flow schedule, and exact EMT-to-kinetic relation are recorded alongside it.

5.1 Raw-Bilinear Analysis Example

The vector channels occur at Gamma positions

$$(X, Y, Z, T) = (3, 5, 7, 1). \quad (55)$$

For the already averaged and volume-normalized primitive, every EMT component can be reconstructed without reading the large source axis:

```
import h5py
import numpy as np

with h5py.File(path, "r") as h5:
    labels = [x.decode() for x in h5["gamma_list"][...]]
    vector = [labels.index(x) for x in ("X", "Y", "Z", "T")]
    D = h5["avg/derivative_bilinear"][...]
    # B axes are [nu,mu,q,flow,t_abs].
    B = np.take(D, vector, axis=0)
    T = 0.5 * (B + np.swapaxes(B, 0, 1))
    np.testing.assert_allclose(T[3,3], h5["avg/Tmunu/T44"][...])
```

Here `T[mu,nu]` has axes `[q,flow,t_abs]` after the two tensor indices. The division by `volume_norm` must not be repeated because it is already included in `avg/derivative_bilinear`. For stochastic-error analysis, read `raw/derivative_bilinear_pervec` in source blocks, construct the same symmetric tensor, and divide the accumulated mean by `volume_norm`. Complete HP vectors belonging to one base must be averaged before treating bases as independent samples.

The same raw basis also supports other observables. First apply `physical_from_pyquda` on the Gamma axis. Symmetrizing the physical `X5,Y5,Z5,T5` derivative channels yields

$$A_{\mu\nu} = \frac{1}{2} \left(L_{\gamma\mu\gamma_5,\nu}^D + L_{\gamma\nu\gamma_5,\mu}^D \right). \quad (56)$$

Selecting the six local tensor channels gives the raw tensor currents; multiply by i only for the Hermitian $i[\gamma_\mu, \gamma_\nu]/2$ convention. A complete portable example, including connected pion and proton axis orders, is in `docs/EMT_gamma_and_raw_bilinears.md`.

5.2 Storage Weight of One Canonical EMTc

All large arrays are uncompressed. Ignoring the small averaged and metadata datasets, the source-scaling payload has the ratio

$$\text{derivative:local:flowed_noise_norm} = 64 : 16 : 1. \quad (57)$$

The embedded ringed kinetic adds one $q = 0$ channel, or $1/N_q$ unit on this scale. In the measured S8T8 file with $N_{\text{eff}} = 8192$, $N_q = 9$, two flow times, and $N_t = 8$, the complete file is 1,531,264,312 bytes:

Part	Bytes	Complete-file share
<code>raw/derivative_bilinear_pervec</code>	1,207,959,552	78.886%
<code>raw/local_bilinear_pervec</code>	301,989,888	19.722%
<code>raw/flowed_noise_norm_pervec</code>	18,874,368	1.233%
<code>derived/ringed/kinetic_pervec</code>	2,097,152	0.137%
All averaged primitives and ten Tmunu datasets	209,664	0.014%
Bookkeeping, basis metadata, and HDF5 overhead	remainder	0.008%

Thus the 64 per-vector derivative channels determine the file size; the ten convenience `avg/Tmunu` datasets are negligible.

5.3 Base-Oriented Production, Checkpoints, and Finalization

The production wrapper has a single shard output path; there is no monolithic production mode. A base noise and all of its hierarchical-probing vectors form the natural complete estimator unit,

$$L_b = \frac{1}{N_{\text{HP}}} \sum_{h=0}^{N_{\text{HP}}-1} L_{b,h}. \quad (58)$$

Inside one base the code writes fixed solve intervals, defaulting to 64 inversions. Thus HP256 normally produces four part files, while HP16 and plain $\mathbb{Z}_4$ produce one. A partial HP part limits memory and transfer size, but is neither a complete estimator nor an independent resume unit.

Part names encode the base, part, and HP interval, for example

`<canonical-stem>.base000003.part0001.hp0064-0127.h5`

Only MPI rank zero writes HDF5. Each part is first written to a unique temporary path and then atomically renamed. After every part of a base closes, rank zero appends one exact `baseXXXXXX` line to a fingerprinted text sample log. Resume reads only this log and performs no HDF5 probe, so a logged base may already have been transferred. If a base is not logged, all of its parts are recomputed and atomically replaced.

Measurement jobs do not update the canonical file. A separate serial finalizer runs where all parts have been collected and requires complete coverage of bases $0, \dots, N_{\text{base}} - 1$. It checks each expected part once while streaming raw datasets into a temporary canonical EMTc file, accumulates the averages without holding the complete source axis in memory, and derives the EMT and embedded kinetic data from the four vector derivative channels. One atomic rename publishes the complete EMTc, so downstream analysis cannot see a new loop without its matching kinetic data. Non-overlapping base ranges may be scheduled independently; overlapping jobs are unsupported.

One runtime detail is essential: evaluating a flowed contraction loads the flowed gauge into QUDA's global resident gauge state. Before inverting the next stochastic source, the code therefore reloads the original unflowed input gauge. The input Python gauge object is unchanged, but omitting this resident-state restore makes later inversions use the wrong gauge and can trigger precision mismatch failures.

The exact quark dataset and attribute names used by downstream scripts are:

```
gamma_list
gamma_pyquda_ids
gamma_matrices
physical_from_pyquda
derivative_directions
raw/local_bilinear_pervec
raw/derivative_bilinear_pervec
raw/flowed_noise_norm_pervec
raw/source_index
raw/base_noise_index
raw/hp_index
avg/Tmunu/T11
avg/Tmunu/T12
avg/Tmunu/T22
avg/Tmunu/T33
avg/Tmunu/T44
avg/local_bilinear
```

avg/derivative_bilinear
 avg/flowed_noise_norm
 derived/ringed/kinetic_pervec
 derived/ringed/kinetic_spacetime
 noise_scheme
 hp_num_vectors
 effective_n_inversions

Important attributes include

Attribute	Meaning
measurement	quark_1pt.
flow_type, flow_epsilon, flow_steps, flow_times	Gradient-flow schedule.
qext	Momentum labels used by downstream analysis.
volume_norm	V_3 , the spatial-volume normalization.
upper_triangle_only	Whether only $\mu \leq \nu$ components are written.
n_vec, n_base_noise, effective_n_inversions	Stochastic source counts.
noise_scheme, hp_num_vectors, hp_ordering	Source-generation scheme.
n_zn, config_num, noise_stream	Counter-noise order, configuration key, and independent
noise_generator, noise_counter_order	Fixed hash algorithm and the fields entering its counter
operator_normalization, renormalization_applied	Whether the saved quark loop has already been renormalized

6 Gluon One-Point Workflow

The gluon workflow is independent of stochastic quark sources. It constructs a flowed gauge-field strength and then measures a spatially Fourier-projected gluon EMT building block.

6.1 Clover Field Strength

For each independent plane (μ, ν) , the code constructs the four oriented clover plaquettes

$$\mu \rightarrow \nu \rightarrow -\mu \rightarrow -\nu, \quad (59)$$

$$\nu \rightarrow -\mu \rightarrow -\nu \rightarrow \mu, \quad (60)$$

$$-\mu \rightarrow -\nu \rightarrow \mu \rightarrow \nu, \quad (61)$$

$$-\nu \rightarrow \mu \rightarrow \nu \rightarrow -\mu. \quad (62)$$

Let $Q_{\mu\nu}(x)$ be their sum. The anti-Hermitian clover field is

$$A_{\mu\nu}(x) = \frac{1}{8} \left[Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x) \right]. \quad (63)$$

The code projects this matrix to the traceless $\mathfrak{su}(3)$ algebra:

$$A_{\mu\nu}^{\text{tl}}(x) = A_{\mu\nu}(x) - \frac{1}{N_c} \text{Tr}_c[A_{\mu\nu}(x)] \mathbf{1}. \quad (64)$$

Finally, the Euclidean field-strength convention used by the implementation is

$$F_{\mu\nu}(x) = -i A_{\mu\nu}^{\text{tl}}(x). \quad (65)$$

Only the six independent planes are computed, and the antisymmetric entries are filled with

$$F_{\nu\mu}(x) = -F_{\mu\nu}(x). \quad (66)$$

The traceless projection is a field-strength projection. It is not a traceless projection of the final EMT tensor.

6.2 Implemented Gluon EMT Loop

The implemented gluon one-point building block is

$$L_{\mu\nu}^g(q, t; t_f) = \frac{2}{V_3} \sum_{\mathbf{x}} \Phi_q(x) \sum_{\substack{\rho=1 \\ \rho \neq \mu, \nu}}^4 \text{Tr}_c[F_{\mu\rho}(x; t_f) F_{\nu\rho}(x; t_f)]. \quad (67)$$

This maps directly to the code in `EMTDisconnectedGluon1pt.flowed_1pt`:

```
for mu in range(4):
    for nu in range(mu, 4):
        tmp = zeros(...)
        for rho in range(4):
            if rho == mu or rho == nu:
                continue
            tmp += contract("...ab,...ba->...", F[mu][rho], F[nu][rho])
        Tmunu_t[mu, nu, :, step, :] += 2.0 * slice_t
Tmunu_t /= Ns3
```

The factor 2 is explicit in the code. The division by V_3 is applied after all spatial Fourier sums have been accumulated.

6.3 Gradient-Flow Schedule for Gluons

As in the quark workflow, step 0 is measured before flow and therefore is the unflowed gauge observable. After each measurement, the gauge field is advanced by Wilson or Symanzik flow:

```
for step in range(Nsteps + 1):
    measure F and Tmunu
    if step == 0:
        flow gauge for 10 substeps of stepsize/10
    elif step < Nsteps:
        flow gauge for 1 step of stepsize
```

The production implementation deliberately keeps this materialize-and-advance schedule. A persistent resident gauge-flow context was tested for Wilson and Symanzik flow with one and ten output steps. It reproduced every saved component exactly, but after separately prewarming both paths its end-to-end speed changed by only a few percent and was not consistently faster in both execution orders. It therefore did not meet the ten-percent integration threshold. This avoids adding resident-state lifetime logic without a stable performance benefit.

7 Gluon HDF5 Layout

The gluon writer is `save_empt_gluon_1pt_hdf5`. Unlike the quark file, there are no stochastic raw per-vector datasets. The output contains only volume-normalized upper-triangle tensor components:

$$\text{Tmunu}/\text{T11}, \text{T12}, \dots, \text{T44} : [N_q, N_{\text{steps}} + 1, N_t]. \quad (68)$$

The canonical gluon loop is source independent,

$$\text{EMTg}/\langle \text{lat} \rangle . \text{EMTg} . \langle \text{cfg} \rangle . \langle \text{ama} \rangle . \langle \text{sm} \rangle . \text{h5},$$

and is reused for all hadron source positions on the configuration. The shared quark and gluon wrappers use the same `EMT_1PT_FLOW_EPSILON`; its production default is (0.207936), so equal flow indices refer to equal flow times. Important attributes include

Attribute	Meaning
<code>measurement</code>	<code>gluon_1pt.</code>
<code>config_num</code>	Gauge configuration encoded by the canonical file.
<code>flow_type</code> , <code>flow_epsilon</code> , <code>flow_steps</code> , <code>flow_times</code>	Gradient-flow schedule.
<code>qext</code> , <code>pf</code> , <code>p_2pt</code>	Momentum labels used by downstream analysis.
<code>volume_norm</code>	V_3 , the spatial-volume normalization.
<code>upper_triangle_only</code>	Whether only $\mu \leq \nu$ components are written.

8 Disconnected Combination with Proton Two-Point Functions

The quark and gluon one-point loops are hadron independent. They become a proton disconnected three-point building block only after being correlated with the proton two-point function on the gauge ensemble. For a proton two-point function $C_2^p(p_f, t)$, define

$$C_{3,\mu\nu}^{p,\text{disc}}(p_f, p_i; t, \tau; t_f) = \langle C_2^p(p_f, t) L_{\mu\nu}(q, \tau; t_f) \rangle_{\text{cfg}} - \langle C_2^p(p_f, t) \rangle_{\text{cfg}} \langle L_{\mu\nu}(q, \tau; t_f) \rangle_{\text{cfg}}, \quad (69)$$

with

$$q = p_f - p_i. \quad (70)$$

The subtraction term removes the vacuum expectation value of the one-point loop. This subtraction must be done at the ensemble-analysis level using the same configuration set as the proton two-point function.

A basic ratio is

$$R_{\mu\nu}^{p,\text{disc}}(t, \tau; t_f) = \frac{C_{3,\mu\nu}^{p,\text{disc}}(p_f, p_i; t, \tau; t_f)}{C_2^p(p_f, t)}. \quad (71)$$

Kinematic projectors, EMT mixing coefficients, and gradient-flow matching factors are applied later in the physics analysis.

9 End-to-End Proton Diagnostic Workflow

The application directory

`application/EMT_disconnected_1pt/perlmutter`

contains a minimal four-step diagnostic workflow:

1. measure the stochastic quark one-point loop;
2. measure the gluon one-point loop;
3. measure the proton two-point function C_2^p ;
4. combine C_2^p and the one-point loops into disconnected diagnostic three-point objects.

For the quark loop, the builder reads `raw/derivative_bilinear_pervec`, reconstructs the symmetric vector EMT, and forms cumulative stochastic averages

$$\bar{L}_{\mu\nu}^{q,N}(q, t; t_f) = \frac{1}{V_3} \frac{1}{N} \sum_{r=1}^N L_{\mu\nu}^{q,(r)}(q, t; t_f), \quad N = 1, \dots, N_{\text{eff}}. \quad (72)$$

This is the quantity used to check whether the stochastic estimator changes smoothly as more random sources are included.

For a single configuration, the builder can only form the unsubtracted proxy

$$C_{3,\mu\nu}^{\text{unsub},N}(t_{\text{sep}}, q, t; t_f) = C_2^p(t_{\text{sep}}) \bar{L}_{\mu\nu}^{q,N}(q, t; t_f). \quad (73)$$

This object is useful for checking shapes, source-count dependence, time alignment, and HDF5 plumbing. It is not the physical disconnected correlator, because the vacuum subtraction in Eq. (69) requires an ensemble average.

With multiple configurations, the builder applies the ensemble subtraction for each cumulative source count:

$$C_{3,\mu\nu}^{\text{disc},N} = \left\langle C_2^p \bar{L}_{\mu\nu}^{q,N} \right\rangle_{\text{cfg}} - \langle C_2^p \rangle_{\text{cfg}} \left\langle \bar{L}_{\mu\nu}^{q,N} \right\rangle_{\text{cfg}}, \quad (74)$$

$$R_{\mu\nu}^{\text{disc},N} = \frac{C_{3,\mu\nu}^{\text{disc},N}}{\langle C_2^p \rangle_{\text{cfg}}}. \quad (75)$$

The gluon loop has no stochastic source axis, so the builder forms the same unsubtracted and vacuum-subtracted objects without a cumulative N index.

The diagnostic merger output contains:

```
C2
quark/source_count
quark/loop_cumulative
quark/C3_unsubtracted_cumulative
quark/C3_disc_cumulative      # only if Ncfg >= 2
quark/R_disc_cumulative      # only if Ncfg >= 2
gluon/loop
gluon/C3_unsubtracted
gluon/C3_disc                 # only if Ncfg >= 2
gluon/R_disc                 # only if Ncfg >= 2
summary/quark_source_count
summary/quark_T44_q0_flow0_loop_norm
summary/quark_T44_q0_flow0_unsub_ratio_proxy
```

10 How to Read Smoke Tests

A single S8T32 test gauge checks implementation conventions, not physics signals. Useful smoke-test checks are:

- the code runs on the intended backend;
- `flow_times` contains step 0 and the requested flow times;
- `derived_empt_upper_triangle_only=True`;
- quark raw datasets have the expected $[N_{\text{eff}}, 16, N_q, N_{\text{steps}} + 1, N_t]$, $[N_{\text{eff}}, 16, 4, N_q, N_{\text{steps}} + 1, N_t]$, and $[N_{\text{eff}}, 2, N_q, N_{\text{steps}} + 1, N_t]$ shapes;
- HP runs have consistent `raw/source_index`, `raw/base_noise_index`, and `raw/hp_index`;
- gluon datasets contain Tmunu/T11 through T44 and no stochastic quark primitive datasets.

11 Recommended Next Algorithmic Upgrade

The current variance-reduction options are ordinary Z_n noise and hierarchical probing. A planned future direction is multiplier-based graph coloring, motivated by

arXiv:2606.00394.

The motivation is that standard HP naturally tests 2^k source counts, whereas multiplier-based coloring can test intermediate coloring budgets. A future scheme should be added as a separate optional source scheme, with HDF5 metadata that clearly distinguishes it from `hierarchical_probing`.